

TOWSON UNIVERSITY
COLLEGE OF GRADUATE STUDIES

AI-ASSISTED ADOPTION OF NIX FLAKES FOR REPRODUCIBLE
DEVELOPMENT ENVIRONMENTS IN COMPUTER SCIENCE
EDUCATION

by

Luis Dale Gascon

A thesis

Presented to the faculty

of Towson University

in partial fulfillment

of the requirements for the degree

Master's of Computer Science

Department of Computer and Information Sciences

Towson University,

Towson, Maryland

21252

December



Student's Name

Luis Dale Gascon

Chairperson, Thesis Committee

Signature

DocuSigned by:

Sajidul Islam

A1834C6028884BD...

Md Sajidul Islam Sajid

Typed name

Note: Please attach a description of the affiliation and credentials of any non- Towson University members of the Committee, and the members' *curriculum vitae*.

Member

Signature

Signed by:

Sidd Kaza

1C79FD345A714C7...

Sidd Kaza

Typed name

Member

Signature

DocuSigned by:

Blair Taylor

F736E4AFF5A443D...

Blair Taylor

Typed name

Approved by

Graduate Program Director

Signature

Signed by:

Ziyang Tang

C6D51D455F89448...

02/02/26

Date

Department Chairperson

Signature

DocuSigned by:

Michael McQuinn

4FA74552EDAD4F1...

02/02/26

Date

☒ Graduate Faculty Status verified for all committee members

Dean of Graduate Studies

Signature

Signed by:

David R Ownby

0109A385FE4043B...

02/03/26

Date

ACKNOWLEDGMENTS

I'd like to thank Dr. Taylor and Dr. Kaza for their support throughout my graduate program and for the opportunities they have presented. I would also like to thank my parents for their belief in my ability to succeed by with their support.

ABSTRACT

AI-ASSISTED ADOPTION OF NIX FLAKES FOR REPRODUCIBLE DEVELOPMENT ENVIRONMENTS IN COMPUTER SCIENCE EDUCATION

Luis Dale Gascon

Inconsistent development environment can slow down software developers and their team. A new hire spends days configuring their systems. Engineers are tasked in resolving environment issues instead of pushing features.

In an educational environment, having students configure their machines for each course can act as a significant blocker to actual learning. An entire lecture period is often sacrificed to the manual and imperative installation of software and dependencies. Personal devices introduce unknown variables such as their operating systems, configuration, or hardware, resulting in students having to be troubleshoot problems specific to their devices that puts an instructor as a role of tech support for devices they might not have experience with.

I propose a solution that leverages agentic LLMs to synthesize a nix environment via nix flakes. We first provide an instructor’s natural language description, then the agent queries an mcp-nixos server to discover valid package names and configuration options from real time data. The output is validated via nix evaluation commands, where the functional nature of nix provides a deterministic success or failure signal. In the event of a failure, the LLM uses the error trace as context to correct itself until a valid, reproducible environment is produced.

TABLE OF CONTENTS

List of Tables	vii
List of Figures	viii
Chapter I. Introduction.....	1
A. Software Configuration Management.....	1
B. Immutable Operating Systems.....	1
C. Nix.....	1
1) Nix Flakes	1
D. Docker	1
1) DevContainers	1
E. Agent Harness.....	1
1) MCP Servers	1
F. Table of Contents, List of Tables, and List of Figures	1
Chapter II. Literature Review.....	2
A. Use of DevContainers to standardize student’s development environment	2
B. Use of LLMs in generating IaC	2
C. CTF Archive	2
D. Use of Nix Flakes for C++ Projects	2
Chapter III. Research Questions	3
Chapter IV. Proposed Solution	4

Chapter V. Testing.....	5
A. Agent flake synthesis testing	5
1) MCP-NixOS.....	5
2) Test Setup	5
a) Antigravity cli	5
b) Opencode with Ollama	5
3) Input Data	5
4) Agent Skill.....	6
a) Sources.....	7
b) Format	8
5) Pass@K	8
B. Comparison between Nix and DevContainer hardware resource usage	8
Chapter VI. Testing Results.....	9
A. Gemini.....	9
B. Qwen 2.5 Coder 14B	9
C. Security Considerations with flakes	9
Chapter VII Conclusion	10
Appendix A Appendix Title Here.....	11
Appendix B Appendix Title Here.....	12
References.....	13

LIST OF TABLES

LIST OF FIGURES

CHAPTER I.

INTRODUCTION

A. Software Configuration Management

B. Immutable Operating Systems

A Linux operating system with a read-only root file system

The concept of read-only configuration

C. Nix

1) *Nix Flakes*: Flakes is an experimental feature

[1]

D. Docker

1) *DevContainers*:

E. Agent Harness

An agent harness is a

[2]

1) *MCP Servers*:

F. Table of Contents, List of Tables, and List of Figures

CHAPTER II.

LITERATURE REVIEW

A. Use of DevContainers to standardize student's development environment

[3] highlights the benefits of DevContainers compared to pre-configured servers that the students remotely connect to. Pre-configured servers introduces friction with instructors as it requires them to collaborate with whoever manages the server. The process can be slow at times since the hosts manages multiple servers.

The students would also need to be on campus or connect via ssh. Having a good and stable connection isn't guaranteed and any drops can hinder the student's learning experience, especially in a graphical environment.

B. Use of LLMs in generating IaC

[4] concludes that LLMs lack awareness of current best practices

C. CTF Archive

To address the issue of students having to manually and imperatively reconstruct the CTF environment, [5] introduced CTF Archive. CTF Archive is a platform to archive ready-to-use environments. These ready-to-use environments comes in the form of docker images that can be easily redeployed

D. Use of Nix Flakes for C++ Projects

[6] talks about their success of migrating away from Docker containers to packaging their lab environments in a nix flake file and hosted it on a public Git repository called fetnix.

CHAPTER III.

RESEARCH QUESTIONS

1. Can an AI agent accurately infer system-level dependencies from natural language assignment instructions or unconfigured source code to automatically generate valid `flake.nix` environments?
2. How does the computational footprints (memory overhead, disk deduplication across heterogeneous course suites, and cold-start latency) of Nix Flakes, and OCI Dev Containers compare?
3. While containerization and declarative environments have solved the 'works on my machine' problem in software engineering, what continues to hinder their adoption in university classrooms?

CHAPTER IV.

PROPOSED SOLUTION

CHAPTER V.

TESTING

A. Agent flake synthesis testing

Answering RQ1 will allow instructors to

1) *MCP-NixOS*: mcp-nixos is a FastMCP server that provides real-time information about the Nix ecosystem. Information could be Home Manager settings,

It leverages search.nix.org Elastic search backend

nixpkgs is a massive repository. At the time of this writing, nixpkgs hosts 139,008 packages across 1,044 packages sets. Not all of the

[7] observes that Rust has the highest package hallucination rates, which suggest that large language models have a higher chance of hallucinating package names due to the scarcity of Rust code compared to Python and Javascript. Nix isn't a popular language

2) *Test Setup*: To automate the tests, we wrote a Python script that invoke system commands

a) *Antigravity cli*: An agent that is readily available to the public

b) *OpenCode with Ollama*: OpenCode has the ability to communicate with an LSP server. The LSP server used in the generation of our flake file

3) *Input Data*: Input data was artificially generated by an LLM. The prompt took inspirations from

We split the data by 2 tracks. Software development and Data Science. Tools between tracks are different enough to warrant a split in tracks.

4) *Agent Skill*: An agent skill is a reusable prompt. It turns a general purpose AI agent into a specialist. A skill may be loaded automatically by an agent based on the skill's description, so it's vital for the author to provide a descriptive skill that matches the

An agent skill may be also manually loaded by the user. In Antigravity CLI, a skill may be invoked by

A skill is a markdown file that contains the following yaml front matter metadata:

- name: Name of the skill
- description: The keywords that agents matches against a user's prompt to determine whether to trigger the skill.

Listing V.1: Course flake generator metadata

```
---
name: course-flake-generator
description: Generates a nix flake file based on an instructor's requirements or with a
  provided project folder with source code and other artifacts. Use when creating, generating
  , or configuring a Nix flake (flake.nix) for course assignments, lab environments, or
  software projects from specifications, natural language instructions, or existing codebases
  . Don't use for non-Nix environments, simple shell scripts, or general documentation
  generation.
---
```

for recurrent behaviors as it improves behavioral consistency.

In the case of generating a flake, creating a skill for

We generated a skill for this purpose, with the assistance of a skill on best practice for creating agent skill with Gemini 3.7 Flash, reasoning set to high.

Listing V.2: custom skill filetree structure

```

.
├── assets
│   ├── flake-devshell.template.nix
│   └── mcp-servers.template.json
├── references
│   ├── flake-patterns.md
│   ├── inferring-requirements.md
│   └── mcp-nixos.md
├── scripts
│   ├── detect-project-requirements.py
│   ├── query-nixos.py
│   └── validate-flake.py
└── SKILL.md

```

According to [8],

With agy, we had to manually refer to the skill via

`/course_flake_generator`.

Before the addition of skills, the agent attempted to install the dependencies from the input data imperatively via brew, as the OS I conducted this test is with a MacBook.

This is a limitation with skills

On prompts focusing on data science tracks, those inputs resulted in a lot of fails, despite specifying the custom skill, the agent opted to install dependencies via brew, resulting in no flake.nix file being generated. Skills aren't guaranteed to be deterministic

a) Sources:

b) Format:

5) Pass@K:

B. Comparison between Nix and DevContainer hardware resource usage

This tests answers RQ2.

Due to data centers strong demand for computer memory, DRAM manufacturers prioritize profitability

CHAPTER VI.

TESTING RESULTS

A. Gemini

Perform a group by for `tool_calls` to track the most common tool calls throughout the tests

Gemini 3.8 Flash set to Medium reasoning

Analyzing the frequency of the different tool calls, the model used the `nix CLI` instead of the Nix MCP server

Gemini's training data likely contains information on popular nix packages used for development

B. Qwen 2.5 Coder 14B

C. Security Considerations with flakes

Abstracting away

CHAPTER VII.

CONCLUSION

APPENDIX A

APPENDIX TITLE HERE

[Appendices follow the last page of the text. Each appendix should be labeled, either at the top or on a proceeding blank page, as Appendix A, Appendix B, etc. The sequence of the appendices follows the order in which they were first introduced in the main body of the text. Each appendix needs to be labeled and named in the main body of the text for it to be included in the appendix section.]

APPENDIX B

APPENDIX TITLE HERE

[Appendices follow the last page of the text. Each appendix should be labeled, either at the top or on a proceeding blank page, as Appendix A, Appendix B, etc. The sequence of the appendices follows the order in which they were first introduced in the main body of the text. Each appendix needs to be labeled and named in the main body of the text for it to be included in the appendix section.]

REFERENCES

- [1] W. Du, J.-M. Gratien, R. Gayno, and B. Raffin. Nix to the Rescue for a Reproducible HPC-AI Software Stack. [Online]. Available: <http://arxiv.org/abs/2607.26688>
- [2] Q. Meng, Y. Wang, L. Chen, Y. Li, W. Wu, W. Jiang, Q. Wang, C. Lu, Y. Gao, Y. Wu, and Y. Hu. Agent Harness for Large Language Model Agents: A Survey. [Online]. Available: <https://www.preprints.org/manuscript/202604.0428>
- [3] S. Valstar, W. G. Griswold, and L. Porter, “Using DevContainers to Standardize Student Development Environments: An Experience Report,” in *Proceedings of the 2020 ACM Conference on Innovation and Technology in Computer Science Education*. ACM, pp. 377–383. [Online]. Available: <https://dl.acm.org/doi/10.1145/3341525.3387424>
- [4] K. G. Srivatsa, S. Mukhopadhyay, G. Katrapati, and M. Shrivastava, “A Survey of using Large Language Models for Generating Infrastructure as Code.”
- [5] P. Gupta, A. Gabani, C. Nelson, and Y. Shoshitaishvili. CTF Archive: Capture, Curate, Learn Forever. [Online]. Available: <http://arxiv.org/abs/2512.01233>
- [6] (5) Building Reproducible, Interactive C++ Learning Environments with Nix Flakes and Pixi | LinkedIn. [Online]. Available: <https://www.linkedin.com/pulse/building-reproducible-interactive-c-learning-nix-flakes-hasanovi%C4%87-tol9e/>

- [7] Importing Phantoms: Measuring LLM Package Hallucination Vulnerabilities.
[Online]. Available: <https://arxiv.org/html/2501.19012v1>

- [8] G. Ling, S. Zhong, and R. Huang. Agent Skills: A Data-Driven Analysis of Claude Skills for Extending Large Language Model Functionality. [Online]. Available: <https://arxiv.org/abs/2602.08004>

